

MODULE 14 · PART 4 - SEQUENCING AND TIME

Sessions and Sequence

Turning an event stream into visits, and the difference between two things happening and one following the other.

PART 4 of 8 MODULE 14 of 26 RUNS about 34 min

What an event stream is not

- ◆ **It is not a table of visits.** It is a table of taps, and nobody recorded where one visit ended and the next began.
- ◆ **It has no natural grain above the event,** so every aggregate above it is a definition somebody invented.
- ◆ **Which is why two teams get two numbers** from one table, and both are right.
- ◆ **The session is the first modelled thing here,** and everything downstream depends on it being one definition.

THE TAKEAWAY

A sales line has a receipt above it, which is a fact about the world. An event has a session above it, which is a fact about a decision somebody made.

A session is **an island in time**. Everything from module thirteen applies, with one predicate changed.

What the raw table holds

1	event_key	member	event_ts	event_type	screen
2	-----+	-----+	-----+	-----+	-----
3	8812041	30417	2026-05-12 09:14:02	login	screen_3
4	8812042	30417	2026-05-12 09:16:41	browse	screen_7
5	8812043	30417	2026-05-12 09:19:08	view	screen_7
6	8812044	30417	2026-05-12 09:21:55	add_to_basket	screen_7
7	8812045	30417	2026-05-12 17:44:10	notification	screen_1
8	8812046	30417	2026-05-12 17:44:52	view	screen_2

7 Eight hours later. Same member, same day, obviously a different visit, and nothing in the row says so.

THE TAKEAWAY

Six rows, two visits, and no column that distinguishes them. Every session number here is derived from the timestamp column alone.

What makes this harder than trading spells

- ◆ **The rule is a convention, not a fact.** A store was genuinely shut. Thirty minutes of silence is somebody's decision.
- ◆ **Changing it moves every downstream number,** and not proportionally. The session count has a knee in it.
- ◆ **The partition is a person,** so there are fifty thousand of them, each with a first event and a last.
- ◆ **And the events are not all equal.** A background notification is not a visit, and filtering it changes the sessions.

THE TAKEAWAY

Trading spells have a fact underneath them. A session has a convention, and the convention is usually undocumented and usually inherited.

Sessionising, in full

```

1 WITH flagged AS (
2   SELECT member_durable_key AS m, event_key, event_ts, event_type,
3         CASE WHEN event_ts - LAG(event_ts) OVER w > INTERVAL 30 MINUTE
4             OR LAG(event_ts) OVER w IS NULL
5             THEN 1 ELSE 0 END AS starts_session
6   FROM fact_app_event
7   WINDOW w AS (PARTITION BY member_durable_key
8               ORDER BY event_ts, event_key)),
9 numbered AS (
10  SELECT *, SUM(starts_session) OVER (PARTITION BY m
11    ORDER BY event_ts, event_key
12    ROWS UNBOUNDED PRECEDING) AS session_no FROM flagged)
13 SELECT m, session_no, MIN(event_ts), MAX(event_ts), COUNT(*)
14 FROM numbered GROUP BY m, session_no;

```

8 The ordering ends in event_key. Two events can share a timestamp, and without it the boundary moves.

THE TAKEAWAY

Module thirteen's flag and running sum, with an interval predicate. 2,599,410 events become 399,822 sessions.

Reading a sessionisation out loud

WRITTEN

SAID

```
PARTITION BY member_durable_key
```

visits belonging to one member

```
ORDER BY event_ts, event_key
```

in the order they happened

```
event_ts - LAG(event_ts) OVER w > INTERVAL 30 MINUTE
```

half an hour of silence ends the visit

```
OR LAG(event_ts) OVER w IS NULL
```

and the member's first ever event starts one

```
SUM(...) OVER (... ROWS UNBOUNDED PRECEDING)
```

how many visits have started so far

THE TAKEAWAY

Row four is the one people omit. Without it the first session of every member is never started, and the numbering begins at zero.

Thirty minutes is a convention. **Measure what it costs you** before defending it.

The threshold curve, measured

INACTIVITY RULE	SESSIONS
5 minutes	786,627
15 minutes	429,394
30 minutes	399,822
60 minutes	399,548
120 minutes	399,201
240 minutes	398,622

THE TAKEAWAY

Flat from 15 minutes to four hours, and then it doubles below 15. The choice of 30 is almost free; the choice of 5 would change every number in the business.

What the knee tells you

- ◆ **The flat region is where the threshold exceeds the within-visit gap.** Here the median gap inside a session is 114 seconds.
- ◆ **Below the knee you are splitting real visits,** because a person pausing to read for six minutes becomes two sessions.
- ◆ **So pick a threshold in the flat region, not at its edge,** and the exact number stops mattering. That is the useful finding.
- ◆ **And if there is no flat region at all,** your events are not visits. Something is generating them on a timer.

THE TAKEAWAY

The last point is the one worth keeping. A session curve with no knee usually means background events, and those are not what anybody means by a visit.

Where the threshold should live

- ◆ **In one named place**, so that changing it is one edit and the effect is measurable.
- ◆ **Not repeated across eleven dashboards**, because then it is eleven definitions and they will drift.
- ◆ **And the sessions themselves belong in a table**, for the same reason module thirteen's spells did.
- ◆ **399,822 rows, from 2,599,410 events**. A six-fold reduction, computed once, read everywhere.

THE TAKEAWAY

A definition that lives in eleven places has eleven values eventually. The session table is the artefact; the threshold is a parameter of it, written down.

Where thirty minutes came from

- ◆ **It is a web analytics default from the 1990s**, chosen when a session meant a sequence of page requests from one browser.
- ◆ **Nothing about it was derived from your users**, or from an app, or from a phone that stays in a pocket.
- ◆ **Which does not make it wrong.** It sits comfortably inside the flat region on this data, which is the only test that matters.
- ◆ **But defend it with the curve, not the convention.** One is evidence and the other is inheritance.

THE TAKEAWAY

Everybody uses thirty minutes and almost nobody can say why. Being able to say 'because the count is flat from fifteen to two hundred and forty' is a better answer.

Two events in one session is not **one event followed by the other**. Almost every funnel confuses them.

Six rows, and the answer by hand

1	event_ts	gap	starts	session_no
2	-----	-----	-----	-----
3	2026-05-12 09:14:02	NULL	1	1
4	2026-05-12 09:16:41	2m 39s	0	1
5	2026-05-12 09:19:08	2m 27s	0	1
6	2026-05-12 09:21:55	2m 47s	0	1
7	2026-05-12 17:44:10	8h 22m	1	2
8	2026-05-12 17:44:52	42s	0	2

7 The only row where the gap exceeds thirty minutes, and the only row after the first that starts a session.

THE TAKEAWAY

Six rows is enough to check any sessionisation. Two members and a tied timestamp make it enough to catch a wrong one.

Testing a sessionisation

- ◆ **Two members, always.** One member cannot catch a missing PARTITION BY, and that defect is otherwise invisible.
- ◆ **A tied timestamp,** because two events can share a second and the ordering needs a tiebreaker to be total.
- ◆ **A gap of exactly the threshold,** which tests whether your comparison is greater-than or greater-than-or-equal.
- ◆ **And a member with one event,** whose entire session is an edge, and who is 8.4% of this data.

THE TAKEAWAY

The third one catches a genuine off-by-one. A gap of exactly thirty minutes either ends the session or does not, and thousands of sessions depend on which.

The session distribution

```
1 events_per_session | sessions
2 -----+-----
3           1 | 33,585    8.4%
4           2 | 33,247
5          3-6 | 133,900
6          7-12 | 166,290
7         13-24 | 32,800
8
9 -- median 6, mean 6.5, max 24
```

3 33,585 single-event sessions, each with a null gap on both sides.

THE TAKEAWAY

Look at this before building on the sessions. A huge spike at one event usually means background events are being counted as visits.

The funnel everybody writes

```
1 SELECT COUNT(*)           AS sessions,  
2       SUM(viewed)         AS viewed,  
3       SUM(viewed * added) AS viewed_and_added,  
4       SUM(viewed * added * checked_out) AS all_three  
5 FROM (  
6     SELECT member_durable_key, session_no,  
7           MAX(CASE WHEN event_type='view'      THEN 1 ELSE 0 END) viewed,  
8           MAX(CASE WHEN event_type='add_to_basket' THEN 1 ELSE 0 END) added,  
9           MAX(CASE WHEN event_type='checkout'  THEN 1 ELSE 0 END) checked_out  
10    FROM sessions GROUP BY member_durable_key, session_no);
```

8 MAX of a CASE asks whether the event happened in the session. It cannot ask whether it happened afterwards.

THE TAKEAWAY

Perfectly readable, widely used, and it answers a different question from the one the word 'then' implies. 399,822 sessions, 120,316 with a view, 38,125 with both.

The funnel with the order in it

```
1 WITH marked AS (  
2     SELECT member_durable_key, session_no, event_ts, event_type,  
3           MIN(CASE WHEN event_type='view' THEN event_ts END) OVER (  
4               PARTITION BY member_durable_key, session_no) AS first_view_ts  
5     FROM sessions)  
6 SELECT COUNT(DISTINCT (member_durable_key, session_no))  
7     FROM marked  
8     WHERE event_type = 'add_to_basket'  
9           AND first_view_ts IS NOT NULL  
10          AND event_ts > first_view_ts;
```

10 The whole difference is this line. The add must happen after the first view, not merely in the same session.

THE TAKEAWAY

21,574 sessions, against 38,125 unordered. The readable funnel overstates this step by 77%.

Why the gap is so large

- ◆ **People add things they already knew they wanted.** An add with no preceding view is an ordinary shopping trip, not a defect.
- ◆ **But the funnel claims a causal story,** and the unordered version supports it with sessions where the order was the other way round.
- ◆ **Both numbers are real.** One is 'sessions with both events' and one is 'sessions where viewing led to adding'.
- ◆ **Only the second is a funnel,** and only the first is usually what got computed.

THE TAKEAWAY

The unordered number is not wrong. It is a different measure wearing a funnel's name, and the name is what makes it misleading.

Once sessions exist, they become **the grain everything else is measured at.**

The session table

```
1 CREATE OR REPLACE TABLE fact_app_session AS
2 SELECT m AS member_durable_key, session_no,
3        MIN(event_ts) AS session_start,
4        MAX(event_ts) AS session_end,
5        MAX(event_ts) - MIN(event_ts) AS duration,
6        COUNT(*) AS events,
7        COUNT(*) FILTER (WHERE event_type='checkout') AS checkouts,
8        MIN(CASE WHEN event_type='view' THEN event_ts END) AS first_view_ts
9 FROM numbered GROUP BY m, session_no;
```

8 Carry the timestamps the funnels will need, once, rather than recomputing them in eleven dashboards.

THE TAKEAWAY

399,822 rows, grain of one row per member per session. Every funnel downstream reads this rather than 2.6 million events.

Bounce rate, three ways

```
1 -- one event in the session
2 COUNT(*) FILTER (WHERE events = 1) * 1.0 / COUNT(*)
3
4 -- session shorter than ten seconds
5 COUNT(*) FILTER (WHERE duration < INTERVAL 10 SECOND) * 1.0 / COUNT(*)
6
7 -- no engagement event of any kind
8 COUNT(*) FILTER (WHERE first_view_ts IS NULL
9                  AND checkouts = 0) * 1.0 / COUNT(*)
```

2 8.4% on Kestrel's data. The other two give different figures from the same sessions.

THE TAKEAWAY

Three definitions, three numbers, one name. Bounce rate is not a measurement, it is a convention, and it is almost never stated alongside the figure.

Every session measure is a definition

- ◆ **Session count depends on the threshold.** Bounce rate depends on what bouncing means. Duration depends on the last event.
- ◆ **Session duration is the worst of them,** because a single-event session has a duration of zero and 8.4% of sessions are that.
- ◆ **So average session duration is dragged down by a third of a population** that arguably has no duration at all.
- ◆ **State the exclusions.** 'Mean duration of sessions with two or more events' is a number somebody can reproduce.

THE TAKEAWAY

Kestrel's mean session duration changes by a large factor depending on whether single-event sessions are in it. Neither answer is wrong and only one of them is stated.

What belongs on a session row

- ◆ **Start, end, duration and event count.** The four every consumer wants and nobody should recompute.
- ◆ **The first timestamp of each event type you report on,** because that is what every ordered funnel compares.
- ◆ **The entry and exit event,** which is first touch and last touch, computed here where the frame gets written correctly once.
- ◆ **Not the conversion flags themselves,** because those are questions, and a new question should not need a schema change.

THE TAKEAWAY

The line is between facts about the session and answers to questions. Timestamps are facts. 'Converted' is an answer, and it changes when somebody redefines conversion.

Kestrel's app team reports a view-to-basket conversion rate of 31.7%, computed as sessions containing both a view and an add-to-basket, divided by sessions containing a view.

A product manager cross-checks against the app's own client-side analytics, which tracks taps in sequence, and gets 17.9%. Both figures are reproducible, and both are computed over the same period and the same events.

Why do they disagree?

- A Duplicate events in the warehouse feed are inflating the numerator.
- B The 30-minute session rule is merging two visits, so unrelated events appear in one session.
- C The warehouse figure tests whether both events occurred, not whether one followed the other.
- D The event types are categorised differently in the two systems, so 'view' means different things.

◆ PAUSE AND DECIDE

Why do they disagree?

A Duplicate events

Would inflate both figures, and the client-side count is the one that is lower. Duplicates also fail a grain assertion, which this passes.

C Presence, not sequence

38,125 sessions contain both events and only 21,574 have the add after the view. That ratio is the discrepancy exactly.

B Sessions merging

Worth checking, and the threshold curve rules it out. The session count is flat from 15 minutes to four hours, so 30 is nowhere near a boundary.

D Different categorisation

A real hazard when comparing two systems, and it would not produce a one-directional gap of this precise size.

THE SIGNAL

"Tracks taps in sequence." The client-side tool measures order because it watches order happen. The warehouse query measures membership, because a GROUP BY has no order in it.

Any ordered question is **a comparison of two timestamps** you carried onto the row.

Carrying the timestamps you need

```
1 SELECT member_durable_key, session_no, event_ts, event_type,  
2      MIN(CASE WHEN event_type='view' THEN event_ts END) OVER s AS first_view,  
3      MIN(CASE WHEN event_type='checkout' THEN event_ts END) OVER s AS first_checkout,  
4      MIN(event_ts) OVER s AS session_start,  
5      LEAD(event_ts) OVER (PARTITION BY member_durable_key, session_no  
6                          ORDER BY event_ts, event_key) AS next_event_ts  
7 FROM sessions  
8 WINDOW s AS (PARTITION BY member_durable_key, session_no);
```

- 3 Conditional aggregation over the session, from module seven. MIN of a CASE gives the first occurrence of one event type.

THE TAKEAWAY

Four timestamps on every row, one sort, one pass. Every ordered question in the rest of this module is a comparison between two of these columns.

Four ordered questions, one shape

- ◆ **Did B follow A?** $b_ts > a_ts$. The funnel step.
- ◆ **How long did it take?** $b_ts - a_ts$. The time to convert.
- ◆ **Did anything happen in between?** An event whose timestamp is strictly between the two.
- ◆ **Did B follow A without C intervening?** $b_ts > a_ts$ AND (c_ts IS NULL OR $c_ts > b_ts$).

THE TAKEAWAY

The fourth is where hand-written SQL gets genuinely hard, and it is what MATCH_RECOGNIZE exists for. Three engines have that, and we come back to it.

Time to next event, and time to convert

```
1 SELECT member_durable_key, session_no,  
2         MEDIAN(EXTRACT(epoch FROM next_event_ts - event_ts)) AS median_gap_s,  
3         MIN(first_checkout - first_view)                      AS time_to_convert  
4 FROM carried  
5 WHERE next_event_ts IS NOT NULL  
6 GROUP BY member_durable_key, session_no;
```

5 The last event of every session has no next event. 399,822 sessions, so 399,822 nulls, and they are not an edge case.

THE TAKEAWAY

Median gap inside a session is 114 seconds on Kestrel's data, and the longest is 1,799 seconds, which is one second under the threshold by construction.

Filtering events **before** sessionising changes the sessions. Filtering after does not.

The same intent, two orders

```
1 -- sessionise the checkouts: gives sessions OF checkouts
2 WITH e AS (SELECT * FROM fact_app_event WHERE event_type='checkout')
3 -- ... then the usual flag and running sum
4
5 -- sessionise everything, then keep sessions that had a checkout
6 WITH s AS ( ... all events, sessionised ... )
7 SELECT * FROM s
8 QUALIFY MAX(CASE WHEN event_type='checkout' THEN 1 ELSE 0 END)
9           OVER (PARTITION BY member_durable_key, session_no) = 1;
```

- 2 Two checkouts forty minutes apart become two sessions here, because nothing in between is left to hold the session open.

THE TAKEAWAY

The first form answers 'runs of consecutive checkouts'. The second answers 'visits during which a checkout happened'. Only the second is what anybody asks for.

Which events hold a session open

- ◆ **Background events are the hard case.** A push notification fires with no user present, and it holds the session open.
- ◆ **Removing them splits sessions,** and keeping them merges visits that were hours apart. Both are defensible.
- ◆ **Decide which events count as activity,** once, and write the list down next to the threshold.
- ◆ **Then filter before sessionising, deliberately,** because that is the one case where filtering early is correct.

THE TAKEAWAY

Two decisions define a session: how long a gap ends it, and which events count as activity. The second is almost never written down anywhere.

An analyst is asked for the number of sessions in which a member checked out. They filter `fact_app_event` to `event_type = 'checkout'`, then apply the 30-minute sessionisation to the result.

The answer comes back as 41,802. A colleague computes it by sessionising the full stream and then keeping sessions containing a checkout, and gets 14,799. Both queries are correct SQL and neither errors.

What happened?

- A The first query counted runs of consecutive checkouts. Sessionise the full stream, then filter the sessions.
- B The threshold should be raised, since checkouts are naturally further apart than ordinary events.
- C COUNT DISTINCT on checkout events gives the same answer more simply, and both queries are overcomplicated.
- D The second query is wrong, because a session containing two checkouts is being counted only once.

◆ PAUSE AND DECIDE

What happened?

A Sessions of checkouts

Removing the other events removed everything that held a session open, so two checkouts in one visit became two sessions.

C Use COUNT DISTINCT

Counts checkout events rather than sessions containing one. A session with three checkouts would count three times.

B Raise the threshold

Treats the symptom by widening the gap until the fragments rejoin. The session boundaries would still be derived from checkouts alone.

D The second is wrong

It is right. A session with two checkouts is one session in which a member checked out, which is what was asked for.

THE SIGNAL

"Both queries are correct SQL." They are, and they answer different questions. Filtering before sessionising changes what a session is, and nothing in the output says so.

First touch and last touch are **FIRST_VALUE** and **LAST_VALUE** over a session. Nothing more.

The funnel in one pass

```
1 SELECT
2     COUNT(*)                                AS sessions,
3     COUNT(*) FILTER (WHERE first_view_ts IS NOT NULL) AS viewed,
4     COUNT(*) FILTER (WHERE first_add_ts > first_view_ts) AS added_after_view,
5     COUNT(*) FILTER (WHERE first_checkout_ts > first_add_ts
6                       AND first_add_ts > first_view_ts) AS checked_out_in_order
7 FROM fact_app_session;
```

5 Each step carries the conditions of the steps before it, or step three counts sessions that skipped step two.

THE TAKEAWAY

Four numbers from 399,822 rows, in one pass, with the ordering baked in. Every comparison is between two columns already on the row.

Three decisions inside a funnel

- ◆ **Strict or loose ordering.** Must the steps be adjacent, or merely in order with anything in between?
- ◆ **First occurrence or any occurrence.** A member who views, adds, then views again has two possible stories.
- ◆ **Within a session, or within a window of time.** A purchase the next morning may still be the same journey.
- ◆ **None of the three is visible in the output,** so all three belong in the measure's name or beside it.

THE TAKEAWAY

Three binary decisions is eight possible funnels, all defensible, all producing different numbers, and all usually called 'conversion rate'.

Time between steps

```
1 SELECT
2     MEDIAN(EXTRACT(epoch FROM first_add_ts - first_view_ts)) AS view_to_add_s,
3     MEDIAN(EXTRACT(epoch FROM first_checkout_ts - first_add_ts)) AS add_to_checkout_s,
4     COUNT(*) AS converted
5 FROM fact_app_session
6 WHERE first_checkout_ts > first_add_ts
7     AND first_add_ts > first_view_ts;
```

6 The filter is the funnel. Without it, these medians include sessions where the steps happened in the wrong order.

THE TAKEAWAY

Timing a step only means anything for sessions that completed it in order. The median over everything includes negative durations that cancel against positive ones.

Drop-off, and where to measure it

- ◆ **Drop-off between steps is one minus the ratio**, and it depends entirely on the denominator you chose.
- ◆ **Of all sessions, or of sessions that reached the previous step?** The second is useful and the first is common.
- ◆ **Kestrel: 120,316 viewed, 21,574 added in order.** That is 82% drop-off from viewers, and 95% of all sessions.
- ◆ **Both are true and they are not the same claim.** Module seven's denominator problem, in a funnel chart.

THE TAKEAWAY

Eighty two percent and ninety five percent, from the same two numbers. The chart usually does not say which denominator was used.

First touch and last touch

```
1 SELECT member_durable_key, session_no, event_ts, event_type, screen,  
2     FIRST_VALUE(screen) OVER w AS first_touch,  
3     LAST_VALUE(screen) OVER w AS last_touch  
4 FROM sessions  
5 WINDOW w AS (PARTITION BY member_durable_key, session_no  
6     ORDER BY event_ts, event_key  
7     ROWS BETWEEN UNBOUNDED PRECEDING  
8     AND UNBOUNDED FOLLOWING);
```

8 The explicit frame, or LAST_VALUE returns the current row. Module five, and this is where it bites in practice.

THE TAKEAWAY

First touch works with or without the frame, and last touch does not. Testing only the first is how this ships broken.

The attribution question nobody settles

- ◆ **First touch of what?** The session, the day, the week before the purchase, or the member's entire history.
- ◆ **Each is a different PARTITION BY**, and each gives a materially different answer for the same purchase.
- ◆ **Session-level attribution ignores the visit three days ago** that actually caused the purchase.
- ◆ **So state the window in the measure's name.** 'First touch within session' is a different column from 'first touch in 7 days'.

THE TAKEAWAY

The partition is the model. Changing it is not a refactor, and a report that does not name its window cannot be compared with one that does.

MATCH_RECOGNIZE runs **a regular expression over rows**. Three engines have it.

MATCH_RECOGNIZE, by engine

ENGINE	ROW PATTERN RECOGNITION
Snowflake, Oracle, Trino	Yes. The three implementations in common use.
Flink	Yes, and it is a streaming engine, not a warehouse.
Amazon Redshift, BigQuery	No.
PostgreSQL	No. A patch was still under discussion in January 2026.
DuckDB, SQL Server, MySQL, Spark	No. DuckDB has had an open request since 2022.

THE TAKEAWAY

Kestrel runs Redshift, Postgres and DuckDB, so this course cannot use it. The window version is the portable answer.

What you lose without it

- ◆ **Patterns with repetition.** A view, then any number of scrolls, then an add. The star is hard to write by hand.
- ◆ **Negation.** B followed A with no C in between. Expressible, and it costs an extra window and a null check.
- ◆ **Simple ordering is fine.** Did B follow A is one timestamp comparison, and that covers most real funnels.
- ◆ **So the gap is narrower than its reputation,** and it is real once the pattern has a repetition or an alternation in it.

THE TAKEAWAY

Most funnels are three ordered steps, which windows handle. The moment somebody says 'and then they browsed for a while', you are writing a state machine by hand.

An analyst reads about `MATCH_RECOGNIZE` and proposes rebuilding Kestrel's funnel reporting on it, on the grounds that it expresses ordered patterns directly rather than through timestamp comparisons.

Kestrel's warehouse is Redshift and its operational replica is PostgreSQL 16. The funnel in question is three ordered steps with no repetition: view, then add to basket, then checkout.

What should the review say?

- ☐ A Approve. `MATCH_RECOGNIZE` is in the SQL standard, so Redshift will support it.
- ☐ B It is Snowflake, Oracle and Trino only. A three-step ordered funnel is two timestamp comparisons, which both engines have.
- ☐ C Move the funnel workload to Snowflake, since pattern matching is the right tool and the engine should follow the need.
- ☐ D Use a recursive CTE to walk the events in order, which expresses the same pattern portably.

◆ PAUSE AND DECIDE

What should the review say?

- A

 The standard implies support
Being in the standard has never implied support. QUALIFY is not in the standard and is widely available; this is in it and is not.
- B

 Not available, and not needed
Three ordered steps with no repetition is two timestamp comparisons. The clause earns its place on patterns with repetition, which this is not.
- C

 Move to Snowflake
A platform migration to avoid writing two predicates. It would be a serious proposal if the patterns were genuinely complex, and they are three steps.
- D

 A recursive CTE
Expressible and far worse. Recursion over two point six million events to answer a question two window functions already answer.

THE SIGNAL

"Three ordered steps with no repetition." That clause is the whole answer. The tool is right for a class of problem this funnel is not in.

Assertions for a session table

- ◆ **Coverage.** The sum of event counts across sessions equals the number of events. Module thirteen's first assertion.
- ◆ **No overlap.** No two sessions for one member share a moment, which a wrong partition key would fail.
- ◆ **No adjacency.** Consecutive sessions are more than the threshold apart, which catches a session that should have been one.
- ◆ **And no internal gap exceeds the threshold**, which is the fourth one and the one specific to time-based islands.

THE TAKEAWAY

Three from module thirteen and one new. The fourth catches a session that should have been split, which the other three cannot see.

The gap between sessions

```
1 SELECT member_durable_key, session_no, session_start,  
2      session_start - LAG(session_end) OVER (  
3          PARTITION BY member_durable_key  
4          ORDER BY session_start) AS since_last_visit  
5 FROM fact_app_session;
```

- 4 LAG over the sessions rather than over the events. The session table is now just another ordered series.
- 5 And every member's first session has a null gap, which is the same edge one level up.

THE TAKEAWAY

Once sessions are a table, every technique in Part 4 applies to them too. Return frequency is gaps-and-islands over sessions rather than over events.

Session measures and member measures

- ◆ **Per session: duration, events, bounce, conversion.** One row per visit, and they average across visits.
- ◆ **Per member: sessions per month, days since last visit, lifetime value.** One row per person.
- ◆ **Averaging a session measure across members double counts the frequent visitors,** which is module seven exactly.
- ◆ **Decide which population the mean is over,** and say so. Mean session duration, or mean duration per member?

THE TAKEAWAY

A member with forty sessions contributes forty rows to a session-level average and one to a member-level one. Both are defensible, and they are different numbers.

Entry and exit screens

```
1 SELECT entry_screen, exit_screen, COUNT(*) AS sessions,  
2        AVG(events) AS mean_events,  
3        COUNT(*) FILTER (WHERE checkouts > 0) * 1.0 / COUNT(*) AS conv  
4 FROM (  
5     SELECT member_durable_key, session_no,  
6            FIRST_VALUE(screen) OVER w AS entry_screen,  
7            LAST_VALUE(screen)  OVER w AS exit_screen  
8     FROM sessions  
9     WINDOW w AS (PARTITION BY member_durable_key, session_no  
10                  ORDER BY event_ts, event_key  
11                  ROWS BETWEEN UNBOUNDED PRECEDING  
12                          AND UNBOUNDED FOLLOWING))  
13 GROUP BY entry_screen, exit_screen;
```

11 The frame again. Without it, exit_screen is the current row's screen and every session appears to exit where it is.

THE TAKEAWAY

Entry and exit by screen, with conversion against each pair. The frame is the only thing standing between this and a report that says every session exits from wherever it is.

The fourth assertion

```
1 SELECT NOT EXISTS (  
2     SELECT 1 FROM (  
3         SELECT member_durable_key, session_no, event_ts,  
4             LAG(event_ts) OVER (PARTITION BY member_durable_key, session_no  
5                                 ORDER BY event_ts, event_key) AS prev_ts  
6         FROM sessions)  
7     WHERE prev_ts IS NOT NULL  
8     AND event_ts - prev_ts > INTERVAL 30 MINUTE  
9 ) AS no_internal_gap_exceeds_threshold;
```

8 The same threshold, restated as a check. If the sessionisation was right, nothing inside a session can exceed it.

THE TAKEAWAY

The threshold appears twice: once building the sessions and once proving them. Writing it as a constant in one place is what keeps the two in step.

A GROUP BY has **no order in it**. If the question contains the word 'then', a GROUP BY cannot answer it alone.

When to leave the session

- ◆ **A journey is often longer than a visit.** Research on Tuesday, purchase on Thursday, and the session model loses the link.
- ◆ **Partition by member rather than by session,** and the same techniques work across the whole history.
- ◆ **But that partition has no natural end,** so first touch means first ever, which is rarely what anybody means.
- ◆ **So bound it with a frame.** Seven days before the purchase, which is a RANGE interval frame from module five.

THE TAKEAWAY

Session-level attribution is easy and often wrong. Member-level attribution is right and needs a stated window, or it reaches back to the member's first ever tap.

A seven-day attribution window

```
1 SELECT member_durable_key, event_ts, event_type, screen,  
2     FIRST_VALUE(screen) OVER (  
3         PARTITION BY member_durable_key  
4         ORDER BY event_ts  
5         RANGE BETWEEN INTERVAL '7 days' PRECEDING  
6         AND CURRENT ROW) AS first_touch_7d  
7 FROM fact_app_event;
```

5 A RANGE interval frame, so the window is seven days rather than a number of events. Module five.

THE TAKEAWAY

Not available on Redshift, which has no RANGE frame at all. On Kestrel's warehouse this becomes a join to a bounded subquery.

What it costs

- ◆ **One sort of the events, by member and timestamp.** Everything else rides on that sort.
- ◆ **The funnel's conditional aggregations are free if they share the partition,** and cost a sort each if they do not.
- ◆ **Materialise the sessions.** 2.6 million events become 399,822 rows, and every funnel then reads the small table.
- ◆ **Which also fixes the definition problem,** because the threshold is then applied once rather than in every report.

THE TAKEAWAY

The performance argument and the correctness argument point the same way here, which is rare. Materialising the sessions gives you one definition and a six-fold reduction at once.

Kestrel has eleven dashboards that sessionise `fact_app_event` independently. Nine use a 30-minute rule, one uses 20 and one uses 60. Nobody knows why the two differ, and both predate the current team.

A funnel figure quoted in a board pack does not match the same figure on a dashboard. The event data is identical and both queries are correct.

What is the first thing to do?

- A Standardise all eleven on 30 minutes, since that is what nine of them already use and it is the industry convention.
- B Add the threshold to each dashboard's title, so that consumers can see which definition produced the number.
- C Measure the session count across thresholds to establish which value is correct before changing anything.
- D Build one session table with the threshold as a named parameter, and have all eleven read it.

◆ PAUSE AND DECIDE

What is the first thing to do?

A Standardise on 30

The right destination reached by a vote. It also leaves eleven copies of the rule, so they drift apart again within a year.

C Measure first

*Worth doing and it answers a question nobody asked.
There is no correct threshold to establish, which is the point of the curve.*

B Label the dashboards

Honest, and it documents the inconsistency rather than removing it. Eleven labelled definitions still disagree in the board pack.

D One session table

Eleven definitions become one, with the threshold named in a single place. The figures then agree because they share a source.

THE SIGNAL

"Both queries are correct." Eleven correct queries producing different numbers is not a SQL problem. It is one definition living in eleven places.

What the session table unlocks

- ◆ **Every funnel, from one small table**, with the ordering already carried as timestamps.
- ◆ **Return frequency and lapse**, by applying Part 4's techniques to the sessions rather than the events.
- ◆ **Entry and exit analysis**, with the frame written correctly once rather than in every report.
- ◆ **And one threshold**, so that eleven dashboards cannot disagree about how many visits there were.

THE TAKEAWAY

Four capabilities from one table of 399,822 rows. The argument for building it is not performance, though it is six times smaller. It is that there is one definition.

Five checks before publishing a funnel

- ◆ **Does each step require the previous one?** Compare the ordered count with the unordered one and see the gap.
- ◆ **What is the denominator of each rate?** All sessions, or sessions that reached the previous step.
- ◆ **Is the window stated?** Within a session, within a day, within seven days. The number changes with each.
- ◆ **Does the timing filter match the funnel filter?** Median time to convert should cover only converting sessions.
- ◆ **And does it reconcile with the client-side tool?** If it does not, one of you is measuring presence.

THE TAKEAWAY

Five questions, and the first one alone changed this module's headline figure by 77%. None of them requires looking at the SQL.

Building a session table, in four steps

01

Activity

Which events count?
Decide, write it down,
and filter before
sessionising.

02

Threshold

Measure the curve, pick
a value in the flat
region, and name the
constant.

03

Sessionise

Module thirteen's flag
and running sum, with
a total ordering.

04

Assert

Coverage, overlap,
adjacency, and no
internal gap above the
threshold.

THE TAKEAWAY

Step one is the one that gets skipped, and it is the only one that changes what a session is rather than how many there are.

MATCH_RECOGNIZE, for reference

```
1 SELECT * FROM events MATCH_RECOGNIZE (  
2   PARTITION BY member_durable_key  
3   ORDER BY event_ts  
4   MEASURES A.event_ts AS viewed_at, C.event_ts AS bought_at  
5   ONE ROW PER MATCH  
6   PATTERN (A B* C)  
7   DEFINE A AS event_type = 'view',  
8           B AS event_type <> 'checkout',  
9           C AS event_type = 'checkout');
```

6 A view, then any number of non-checkouts, then a checkout. That star is what windows cannot express cleanly.

THE TAKEAWAY

Snowflake, Oracle and Trino. Worth being able to read, because you will meet it in documentation long before you can run it.

Reading it without running it

- ◆ **PATTERN** is a regular expression over rows. A B^*C means one A, any number of Bs, then one C.
- ◆ **DEFINE** gives each letter a predicate. A row is an A if it satisfies A's condition.
- ◆ **MEASURES** projects from the matched rows, so A.event_ts is the timestamp of the row that matched A.
- ◆ **ONE ROW PER MATCH** collapses the match, where ALL ROWS PER MATCH keeps every row in it.

THE TAKEAWAY

Four clauses, and it reads like a regex once you know that. You will meet it in documentation and in migration proposals long before an engine you use supports it.

The confusions, settled

SYMPTOM	THE CAUSE
The funnel rate is far higher than the client's	Presence was tested, not sequence.
Far more sessions than expected	The stream was filtered before sessionising.
Last touch equals the current event	No frame. Module five, arriving in an attribution report.
Two dashboards disagree on session count	Two thresholds, in two places, both undocumented.
Session boundaries move between runs	The ordering has no tiebreaker and timestamps tie.
MATCH_RECOGNIZE is rejected	Correct. Snowflake, Oracle and Trino only.

THE TAKEAWAY

The first row is the expensive one, because it produces a number that is wrong in a flattering direction and therefore gets quoted.

Five habits

- ◆ **Measure the threshold curve once**, and pick a value in the flat region rather than defending a convention.
- ◆ **Sessionise the whole stream, then filter sessions**. The only exception is deciding which events are activity.
- ◆ **Put the sequence in the funnel**, because presence and order differ by 77% on this data and always in the flattering direction.
- ◆ **Write the frame on FIRST_VALUE and LAST_VALUE**, because one of them works without it and the other does not.
- ◆ **Materialise the sessions**, so the threshold is applied once and every downstream number agrees.

THE TAKEAWAY

The third habit is the one that changes a published figure. The other four change how reliably you can defend it.

A session is an interval. **Module fifteen** is what happens when intervals meet each other.